Project Documentation: Real-Time IoT Data Stream Processing and Aggregation System

Overview

This project consists of three primary components:

1. **IoT Sensor Module**: A real-time IoT module written in Scala with Akka that produces sensor readings and pushes them to a Kafka topic at a rate of 10 ms per message.
2. **Spark Jobs Application**: A Spark application that consumes the sensor readings from Kafka using Spark Streaming, aggregates them, stores them in a Google Cloud Storage (GCS) bucket in Protobuf format, and uploads an aggregated JSON file.
3. **Akka HTTP Application**: An Akka HTTP service that exposes endpoints for querying the aggregated sensor readings.

Key Technologies

- **Scala** and **Akka** for the IoT sensor module.
- **Kafka** for message streaming.
- **Apache Spark** for stream processing and aggregation.
- **Google Cloud Storage (GCS)** for data storage.
- **Protobuf** for efficient binary data serialization.
- **Akka HTTP** for exposing RESTful APIs.

1. IoT Sensor Module (Akka Producer)

Purpose

The IoT module generates sensor readings and sends them to a Kafka topic in real-time. The system is designed to produce messages at a very high frequency (every 10 milliseconds).

Components:

- **Akka Actors**: Used for concurrent, non-blocking operations to handle the high frequency of message production.
- **Kafka Producer**: Pushes the sensor readings to a Kafka topic.

Data Flow:

- Sensor readings (e.g., temperature, humidity, pressure, etc.) are generated at 10 ms intervals.
- These readings are encapsulated in a message and sent to the Kafka topic.

Kafka Topic Configuration:

- **Topic Name**: `sensor-readings`
- **Partitioning**: Based on sensor ID or timestamp to allow for parallelism.

Example Message Structure:

```
{
  "sensor_id": "sensor_001",
  "timestamp": "2024-12-16T12:01:01.001Z",
  "temperature": 22.5,
  "humidity": 55.3,
  "pressure": 1013.25
}
```

Configuration:

- **Akka Dispatcher**: To handle high-throughput asynchronous tasks.
- **Kafka Producer Settings**: Configure Kafka producer for high-throughput publishing.

2. Spark Jobs Application (Stream Processing and Aggregation)

Purpose

The Spark application consumes the sensor readings from the Kafka topic in real-time, performs necessary aggregations, and stores the results in a GCS bucket partitioned by date and hour. Additionally, the aggregated data is uploaded as a JSON file for easy access.

Components:

- **Spark Streaming**: To consume real-time data from Kafka.
- **GCS Sink**: To store aggregated data in a partitioned format (Protobuf) and upload JSON files with the aggregated data.
- **Aggregation Logic**: Aggregates the sensor data in batches triggered every 5 seconds.

Data Flow:

- **Batch Interval**: Spark streaming processes data in 5-second intervals.
- **Aggregation**: Data is aggregated over time by calculating statistics like average temperature, humidity, etc., over the past 5-second window.
- **Partitioning**: The aggregated data is written to GCS in a partitioned directory format: `YYYY/MM/DD/HH/`.
- **Protobuf Format**: Data is stored in Protobuf format for efficient storage.
- **JSON File**: The aggregated data is also stored as a JSON file for easy access and analysis.

GCS Directory Structure:

```
gs://my-bucket/sensor-data/YYYY/MM/DD/HH/
```

Batch Processing Example:

- **Batch Interval**: 5 seconds.
- **Aggregation**: For each batch, calculate the mean or sum of sensor readings over the past 5 seconds.

Sample Aggregated Data (Protobuf):

```
message AggregatedData {
  string sensor_id = 1;
  int32 temperature_sum = 2;
  int32 humidity_sum = 3;
  int32 pressure_sum = 4;
  int32 count = 5;
  string timestamp = 6;
}
```

Example JSON File (Aggregated Data):

```
{
  "sensor_id": "sensor_001",
  "aggregated_readings": {
    "temperature": 22.4,
    "humidity": 54.8,
    "pressure": 1013.00
  },
  "timestamp": "2024-12-16T12:01:05.000Z"
}
```

Spark Application Configuration:

- **Kafka Source**: Reading from Kafka topic `sensor-readings`.
- **Windowing**: Using a 5-second window for aggregations.
- **Protobuf and JSON Formats**: Writing data to GCS in Protobuf format and uploading a JSON summary.

3. Akka HTTP Application (API Service)

Purpose

The Akka HTTP service exposes endpoints that allow users to query the aggregated sensor readings stored in GCS.

Components:

- **Akka HTTP**: Handles HTTP requests and provides endpoints for querying the aggregated data.
- **GCS Access**: The service fetches the aggregated data stored in GCS.
- **JSON Response**: Aggregated data is returned as a JSON response for easy consumption by clients.

API Endpoints:

1. Get Aggregated Data

- **URL**: `/aggregated-data`
- **Method**: `GET`
- **Parameters**:
  - `sensor_id`: The ID of the sensor.
  - `start_date`: The start date for the query (optional).
  - `end_date`: The end date for the query (optional).
- **Response**: Returns aggregated sensor readings in JSON format.

Example Request:

```
GET /aggregated-data?sensor_id=sensor_001&start_date=2024-12-16T12:00:00Z&end_date=2024-12-16T12:05:00Z
```

Example Response:

```
{
  "sensor_id": "sensor_001",
  "aggregated_readings": {
    "temperature": 22.4,
    "humidity": 54.8,
    "pressure": 1013.00
  },
  "timestamp": "2024-12-16T12:01:05.000Z"
}
```

2. Get All Sensor Aggregated Data

- **URL**: `/aggregated-data/all`
- **Method**: `GET`
- **Response**: Returns all aggregated data for all sensors.

Example Request:

```
GET /aggregated-data/all
```

Example Response:

```
[
  {
    "sensor_id": "sensor_001",
    "aggregated_readings": {
      "temperature": 22.4,
      "humidity": 54.8,
      "pressure": 1013.00
    },
    "timestamp": "2024-12-16T12:01:05.000Z"
  },
  {
    "sensor_id": "sensor_002",
    "aggregated_readings": {
```

```
      "temperature": 24.1,
      "humidity": 52.0,
      "pressure": 1014.20
    },
    "timestamp": "2024-12-16T12:01:05.000Z"
  }
]
```

Akka HTTP Server Configuration:

- **Route Definitions**: Define routes to handle HTTP requests and process them accordingly.
- **GCS Access**: Use Google Cloud SDK to fetch the aggregated data from GCS.
- **Serialization**: JSON is used to return aggregated data in a readable format.

System Workflow

Step-by-Step Process:

1. **IoT Sensor Module** generates sensor data every 10 ms and sends it to Kafka.
2. **Spark Jobs Application** consumes the data from Kafka in 5-second batches, aggregates it, and stores it in GCS in Protobuf format. It also generates a JSON file for easy access.
3. **Akka HTTP Application** provides a RESTful API to retrieve aggregated sensor data stored in GCS.

System Configuration

Kafka:

- **Topic**: `sensor-readings`
- **Partitioning Key**: `sensor_id` or `timestamp`

Spark Streaming:

- **Batch Interval**: 5 seconds

- **GCS Path**: `gs://my-bucket/sensor-data/YYYY/MM/DD/HH/`
- **File Format**: Protobuf for storage, JSON for uploading

Akka HTTP:

- **API Routes**: `/aggregated-data`, `/aggregated-data/all`
- **Data Fetching**: Retrieves aggregated data from GCS and serves it in JSON format.

Monitoring and Scaling

Monitoring:

- **Akka**: Use Akka Monitoring and Logging to track message production and consumption rates.
- **Spark**: Use Spark UI to monitor streaming jobs and performance.
- **GCS**: Monitor storage usage to ensure that data is written correctly.

Scaling:

- **Kafka**: Scale Kafka brokers to handle higher throughput.
- **Spark**: Scale the number of executors and workers to handle increased batch data.
- **Akka HTTP**: Horizontal scaling of Akka HTTP servers for better API handling.

Results:

Sensor Readings:

```
{"humidity":22.707874,"sensorId":"cf0e4352-6156-4fc6-86e6-c1aa23ed933a","temperature":2.0105133,"timestamp":1734183436645}
{"humidity":10.180944,"sensorId":"3efaa768-fd6e-41e7-8684-de7ffa0ccef5","temperature":135.7703,"timestamp":1734183437665}
{"humidity":98.717186,"sensorId":"a57dfe74-e763-40f2-89bc-1c47c9cf721d","temperature":-8.026146,"timestamp":1734183438676}
{"humidity":79.826935,"sensorId":"3821cca3-c460-4a60-94fe-ba9e6a3d1de5","temperature":-21.957756,"timestamp":1734183439695}
{"humidity":85.03808,"sensorId":"75abb192-db7f-408e-a87c-1961a5651603","temperature":-36.596725,"timestamp":1734183440716}
{"humidity":31.451744,"sensorId":"f182b9c1-f86d-48a0-95d4-f6ebaaea13a7","temperature":-12.159504,"timestamp":1734183441725}
{"humidity":66.93189,"sensorId":"6e5eae9a-9e06-48ec-8f52-d381fc2384fd","temperature":-49.26902,"timestamp":1734183442746}
{"humidity":66.45687,"sensorId":"8580deda-23c9-4433-afb7-730b3f9d5eb4","temperature":-10.896278,"timestamp":1734183443765}
{"humidity":54.52777,"sensorId":"3829fc82-0b89-47a9-8221-2f2ad5689043","temperature":35.345306,"timestamp":1734183444775}
{"humidity":85.421196,"sensorId":"805ac8d7-3438-4d02-a11a-c70c6ed6e497","temperature":103.37184,"timestamp":1734183445795}
{"humidity":92.02029,"sensorId":"8a792bbb-af17-4048-9601-736fca17fbdd","temperature":-4.0955544,"timestamp":1734183446805}
{"humidity":39.443443,"sensorId":"a672e79a-4aba-4af9-a97a-f970449ce61d","temperature":144.93633,"timestamp":1734183447825}
{"humidity":48.60351,"sensorId":"1336f53b-493f-4988-afdc-1fad21800f0a","temperature":27.035698,"timestamp":1734183448835}
{"humidity":93.45907,"sensorId":"a52154b4-5107-4e98-bc65-c54e2deb9717","temperature":106.9621,"timestamp":1734183449856}
{"humidity":80.19545,"sensorId":"1304e31a-b9d1-4e95-b9c7-543141ab6005","temperature":91.42699,"timestamp":1734183450875}
{"humidity":80.283424,"sensorId":"8c89c53c-7ebc-4b62-94d1-20afa83a4ece","temperature":-18.094276,"timestamp":1734183451894}
{"humidity":86.45948,"sensorId":"7ad8116b-7fc5-4773-a7c2-5f5c268e2c1b","temperature":65.151665,"timestamp":1734183452915}
{"humidity":5.4511013,"sensorId":"2014612f-8fd1-4294-b463-74842667426a","temperature":-7.4706306,"timestamp":1734183453934}
{"humidity":98.82047,"sensorId":"c8e46f9e-813d-4984-a180-b04f410b976e","temperature":114.45915,"timestamp":1734183454955}
{"humidity":87.40871,"sensorId":"67c80cca-d41d-48ef-aec4-14a2afa25529","temperature":22.533585,"timestamp":1734183455965}
{"humidity":94.85708,"sensorId":"96eb8474-fd4b-4713-8fe6-40be4c9f5fc5","temperature":-49.499428,"timestamp":1734183456983}
{"humidity":74.43654,"sensorId":"ffca7d84-6f88-42cc-9bac-faba1091c391","temperature":-11.344383,"timestamp":1734183458006}
{"humidity":54.44395,"sensorId":"dfee8ad2-efb1-484a-b279-0554e98f2f7d","temperature":83.457596,"timestamp":1734183459026}
{"humidity":59.545433,"sensorId":"28aa2de4-57f8-41da-a91b-59f5be1eeec8","temperature":-35.01973,"timestamp":1734183460045}
{"humidity":29.19789,"sensorId":"074765d7-1a69-4e2d-91a5-83ee8ace38fc","temperature":16.838242,"timestamp":1734183461065}
{"humidity":34.80429,"sensorId":"5ded6883-cd95-4a5f-a301-fe51eebc7661","temperature":96.78882,"timestamp":1734183462075}
{"humidity":54.09137,"sensorId":"16521815-2b7b-4c0a-b9ff-280e28a43552","temperature":-45.535076,"timestamp":1734183463096}
{"humidity":55.948704,"sensorId":"67f0420e-3483-4e70-a3d9-110f4d79657d","temperature":-17.245125,"timestamp":1734183464113}
{"humidity":67.539085,"sensorId":"fa4d802b-a124-4697-8655-bde00dcc977d","temperature":145.6925,"timestamp":1734183465125}
{"humidity":72.11197,"sensorId":"83b923e8-4dbf-40b4-90eb-06081299f52b","temperature":-16.464497,"timestamp":1734183466145}
{"humidity":41.90522,"sensorId":"62984c3e-d541-4a06-83f6-eff3b20c4504","temperature":68.07314,"timestamp":1734183467165}
{"humidity":61.07908,"sensorId":"7c0a8507-4b00-415d-9b3c-92e4a5e09bc8","temperature":70.27893,"timestamp":1734183468185}
{"humidity":42.34925,"sensorId":"ada4f948-b1eb-4f57-bbf1-46b13355f6eb","temperature":-18.929756,"timestamp":1734183469195}
{"humidity":71.127785,"sensorId":"6b74a6c5-4565-4ac2-a524-aed9b393a8df","temperature":-37.62331,"timestamp":1734183470215}
{"humidity":99.41815,"sensorId":"94c9c09b-24f0-4a41-84eb-90c2cc2412e0","temperature":110.177795,"timestamp":1734183471226}
{"humidity":31.385237,"sensorId":"7c2b551d-70d6-41c8-a2d9-b1d6fcb73ea7","temperature":39.656464,"timestamp":1734183472246}
{"humidity":85.056526,"sensorId":"ef05d6aa-5919-49d5-a21f-e3f325ab46f6","temperature":9.215118,"timestamp":1734183473265}
{"humidity":55.35499,"sensorId":"684cf8b6-85c4-4aa9-9360-54936a8149b2","temperature":135.49243,"timestamp":1734183474285}
{"humidity":47.05998,"sensorId":"dccc0a19-6a01-4586-b016-e3a59a7e217d","temperature":-38.44739,"timestamp":1734183475305}
```


Raw Sensor Data in GCS:

Buckets > de_case_study_bucket > raw > sensor-data > 2024 > 12 > 14 > 19

CREATE FOLDER UPLOAD TRANSFER DATA OTHER SERVICES

Filter by name prefix only Filter Filter objects and folders Show Live objects only

Name	Size	Type	Created
_SUCCESS	0 B	application/octet-stream	Dec 14, 2024, 7:07:35 PM
part-00000-1ca3535e-dfc4-4288-96	717 B	application/octet-stream	Dec 14, 2024, 7:01:26 PM
part-00000-29d3c8c8-2811-4e98-8	3.2 KB	application/octet-stream	Dec 14, 2024, 7:06:29 PM
part-00000-3c459104-12f8-4a40-8	1.1 KB	application/octet-stream	Dec 14, 2024, 7:06:10 PM
part-00000-757673bb-7f00-4f36-b9	1.1 KB	application/octet-stream	Dec 14, 2024, 7:03:28 PM
part-00000-8e24e71c-2cad-4894-af	1,012 B	application/octet-stream	Dec 14, 2024, 7:02:48 PM
part-00000-990e84ec-6319-424e-b	793 B	application/octet-stream	Dec 14, 2024, 7:01:05 PM
part-00000-b39a4340-a8e3-4c4c-a	1.1 KB	application/octet-stream	Dec 14, 2024, 7:01:45 PM
part-00000-da9657ee-812e-40b4-9	714 B	application/octet-stream	Dec 14, 2024, 7:07:32 PM
part-00000-f7e10ecb-0b99-4c29-9f	2.1 KB	application/octet-stream	Dec 14, 2024, 7:03:09 PM

Aggregated Data in GCS:

Buckets > de_case_study_bucket > aggregated > protobuf > 2024 > 12 > 14 > 19

CREATE FOLDER UPLOAD TRANSFER DATA OTHER SERVICES

Filter by name prefix only | Filter | Filter objects and folders | Show Live objects only

Name	Size	Type	Created
aggregated-data-2024-12-14-19-_S	0 B	application/octet-stream	Dec 16, 2024, 9:00:38 AM
aggregated-data-2024-12-14-19-par	717 B	application/octet-stream	Dec 16, 2024, 9:00:38 AM
aggregated-data-2024-12-14-19-par	793 B	application/octet-stream	Dec 16, 2024, 9:00:38 AM

JSON Stored in GCS:

CREATE FOLDER UPLOAD TRANSFER DATA OTHER SERVICES

Filter by name prefix only | Filter | Filter objects and folders | Show Live objects only

Name	Size	Type	Created	Storage class	Last modified	Pu
_SUCCESS	0 B	application/octet-stream	Dec 12, 2024, 5:18:05 AM	Standard	Dec 12, 2024, 5:18:05 AM	No
part-00000-2f197046-065a-4e22-...	0 B	application/octet-stream	Dec 12, 2024, 5:17:38 AM	Standard	Dec 12, 2024, 5:17:38 AM	No
part-00008-2f197046-065a-4e22-...	189 B	application/octet-stream	Dec 12, 2024, 5:17:43 AM	Standard	Dec 12, 2024, 5:17:43 AM	No
part-00011-2f197046-065a-4e22-...	189 B	application/octet-stream	Dec 12, 2024, 5:17:48 AM	Standard	Dec 12, 2024, 5:17:48 AM	No
part-00041-2f197046-065a-4e22-...	195 B	application/octet-stream	Dec 12, 2024, 5:17:52 AM	Standard	Dec 12, 2024, 5:17:52 AM	No
part-00127-2f197046-065a-4e22-...	192 B	application/octet-stream	Dec 12, 2024, 5:17:55 AM	Standard	Dec 12, 2024, 5:17:55 AM	No
part-00169-2f197046-065a-4e22-...	188 B	application/octet-stream	Dec 12, 2024, 5:17:59 AM	Standard	Dec 12, 2024, 5:17:59 AM	No

Postman Results:

GET http://localhost:8080/api/aggregated-data

Params Authorization Headers (8) Body Pre-reque

Type No Auth

Body Cookies Headers (4) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "avgHumidity": 19.446272,
        "avgTemperature": -17.510841,
        "dataCount": 1,
        "maxHumidity": 19.446272,
        "maxTemperature": -17.510841,
        "minHumidity": 19.446272,
        "minTemperature": -17.510841,
        "sensorId": "Sensor-12"
    },
    {
        "avgHumidity": 12.964559,
        "avgTemperature": 63.142204,
        "dataCount": 1,
        "maxHumidity": 12.964559,
        "maxTemperature": 63.142204,
        "minHumidity": 12.964559,
        "minTemperature": 63.142204,
        "sensorId": "Sensor-76"
    },
```


Conclusion

This system provides a robust pipeline for processing and aggregating IoT sensor data in real-time. Using Akka for message production, Spark for stream processing, and Akka HTTP for exposing the data, it ensures high-throughput data handling, efficient storage in GCS, and easy access via RESTful APIs.